

WADF105 PRACTICAL LABORATORY REPORT

Lab 02: OPNsense Policy Logging and Packet Analysis

Name	Maryjudith Chidinma Ogunaka
Registration Number	C11/26/FCDF/17151
Course	WADF105 Network Security Fundamentals
Programme	ICDFA Fellowship in Cybersecurity and Digital Forensics
Cohort	11
Laboratory	Lab 02 - OPNsense Policy Logging and Packet Analysis
Date	October 2026

All activities were performed in an authorised, isolated virtual laboratory environment.

1. Introduction

This laboratory covers the implementation, testing and review of firewall security policies on an OPNsense firewall. Rules were created to block specific traffic, their effect was validated from a Linux client, and firewall logging was reviewed to understand how policy enforcement affects network communications.

2. Aim

To implement and verify firewall policies in OPNsense, observe their effect on network traffic, review firewall log information, and understand how policy enforcement influences communications.

3. Laboratory Environment

Component	Details
Firewall	OPNsense 26.7 (amd64), icdfa-nslab-firewall-v1.icdfa.test
Client	Ubuntu, icdfa-nslab-client-v1, 10.10.10.157/24
Router	Ubuntu VRouter (WAN gateway 172.16.100.1)
LAN	10.10.10.0/24 (firewall 10.10.10.1)
WAN	172.16.100.2/30

OPNsense acted as the gateway and security device; the Ubuntu client generated the test traffic.

```
You can now access the web GUI by opening
the following URL in your web browser:

https://10.10.10.1

*** icdfa-nslab-firewall-v1.icdfa.test: OPNsense 26.7 (amd64) ***

LAN (em1)    -> v4: 10.10.10.1/24
WAN (em0)    ->

HTTPS: SHA256 EB 0B 30 3E D5 09 EB 63 A0 D0 03 2F E6 38 4C 22
      02 2C 5A A4 39 D5 45 54 57 50 A0 73 50 E0 46 90
SSH:   SHA256 K10LIdmW484dUmuPy37d714RA0De5LKjSU/+bKB1Tw (ECDSA)
SSH:   SHA256 5wuyP8AdB895sP0fHz7CgfNStFMhPdXX42x23cBxc04 (ED25519)
SSH:   SHA256 6e/0XrJSdfl/Hsy8atZw0IPQ6sb1P6zFfuJ/PZ50d+A (RSA)

0) Logout          7) Ping host
1) Assign interfaces 8) Shell
2) Set interface IP address 9) pTop
3) Reset the root password 10) Firewall log
4) Reset to factory defaults 11) Reload all services
5) Power off system 12) Update from console
6) Reboot system 13) Restore a backup

Enter an option: 
```

Figure 1: Firewall console before restore

```
*** icdfa-nslab-firewall-v1.icdfa.test: OPNsense 26.7 (amd64) ***

LAN (em1)    -> v4: 10.10.10.1/24
WAN (em0)    -> v4: 172.16.100.2/30

SSH:   SHA256 K10LIdmW484dUmuPy37d714RA0De5LKjSU/+bKB1Tw (ECDSA)
SSH:   SHA256 5wuyP8AdB895sP0fHz7CgfNStFMhPdXX42x23cBxc04 (ED25519)
SSH:   SHA256 6e/0XrJSdfl/Hsy8atZw0IPQ6sb1P6zFfuJ/PZ50d+A (RSA)

0) Logout          7) Ping host
1) Assign interfaces 8) Shell
2) Set interface IP address 9) pTop
3) Reset the root password 10) Firewall log
4) Reset to factory defaults 11) Reload all services
5) Power off system 12) Update from console
6) Reboot system 13) Restore a backup

Enter an option: 8

root@icdfa-nslab-firewall-v1:~# ifconfig | grep "inet "
inet 172.16.100.2 netmask 0xfffffff0 broadcast 172.16.100.3
inet 10.10.10.1 netmask 0xfffffff0 broadcast 10.10.10.255
inet 127.0.0.1 netmask 0xff000000

root@icdfa-nslab-firewall-v1:~# 
```

Figure 2: Firewall LAN/WAN restored

```
icdfa@icdfa-nslab-client-v1:~$ ip -4 -br address
ip        address      127.0.0.1
lo        127.0.0.1
eth0      10.10.10.157/24
icdfa@icdfa-nslab-client-v1:~$ 
```

Figure 3: Client IP verification

```
icdfa@icdfa-nslab-client-v1:~$ ping -c 4 -b 64 10.10.10.1
ping: 10.10.10.1 (10.10.10.1): 64 bytes of data to 10.10.10.1: 0% loss=0.0%
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=0.44 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.42 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.43 ms
64 bytes from 10.10.10.1: icmp_seq=4 ttl=64 time=0.33 ms

--- 10.10.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 348ms
rtt min/avg/max/mdev = 2.837/0.337/0.352/0.04 ms
icdfa@icdfa-nslab-client-v1:~$ 
```

Figure 4: Client to firewall ping

4. Procedure

1. Verified client addressing and reachability of the firewall.
2. Established a baseline for ICMP, DNS, HTTP and HTTPS.
3. Created the rule **LAB2 BLOCK ICMP TO 1.1.1.1**.
4. Applied changes and tested ICMP.
5. Verified DNS and HTTPS were unaffected.
6. Created the rule **LAB2 BLOCK OUTBOUND HTTP** (TCP/80).
7. Tested HTTP and HTTPS.
8. Reviewed Firewall > Log Files > Live View.
9. Disabled both lab rules to restore connectivity.

5. Results

5.1 Baseline Connectivity

Test	Command	Result	Status
ICMP	ping -c 4 1.1.1.1	4/4 received, 0% loss	PASS
DNS	getent hosts example.com	Resolved (IPv6 records shown)	PASS
HTTP	curl -I http://example.com	HTTP/1.1 200 OK	PASS
HTTPS	curl -I https://example.com	HTTP/2 200	PASS

```
root@kali:~# ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data:
64 bytes from 1.1.1.1: icmp_seq=1 ttl=64 time=0.021 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=64 time=0.020 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=64 time=0.019 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=64 time=0.018 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 300ms
rtt min/avg/max/mdev = 0.044/0.020/0.028/0.001 ms
root@kali:~#
```

Figure 5: Baseline ICMP

```
root@kali:~# ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data:
64 bytes from 1.1.1.1: icmp_seq=1 ttl=64 time=0.022 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=64 time=0.021 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=64 time=0.019 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=64 time=0.018 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 300ms
rtt min/avg/max/mdev = 0.044/0.020/0.028/0.001 ms
root@kali:~# getent hosts example.com
2001:4860:4860::8888 example.com
2001:4860:4860::8888 example.com
root@kali:~#
```

Figure 6: Baseline DNS

```
root@kali:~# ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data:
64 bytes from 1.1.1.1: icmp_seq=1 ttl=64 time=0.021 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=64 time=0.020 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=64 time=0.019 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=64 time=0.018 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 300ms
rtt min/avg/max/mdev = 0.044/0.020/0.028/0.001 ms
root@kali:~# getent hosts example.com
2001:4860:4860::8888 example.com
2001:4860:4860::8888 example.com
root@kali:~# curl -I http://example.com
HTTP/1.1 200 OK
Date: Wed, 07 Oct 2020 12:48:24 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Server: cloudflare
last-modified: Fri, 02 Oct 2020 10:11:02 GMT
Accept-Ranges: bytes
ETag: "1713"
CF-Cache-Status: HIT
CF-RAY: 60c70817a7b26a5d
alt-svc: h2=":443"; ma=86400
root@kali:~#
```

Figure 7: Baseline HTTP

```
root@kali:~# ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data:
64 bytes from 1.1.1.1: icmp_seq=1 ttl=64 time=0.021 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=64 time=0.020 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=64 time=0.019 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=64 time=0.018 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 300ms
rtt min/avg/max/mdev = 0.044/0.020/0.028/0.001 ms
root@kali:~# getent hosts example.com
2001:4860:4860::8888 example.com
2001:4860:4860::8888 example.com
root@kali:~# curl -I https://example.com
HTTP/2 200
Date: Wed, 07 Oct 2020 12:48:24 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Server: cloudflare
last-modified: Fri, 02 Oct 2020 10:11:02 GMT
Accept-Ranges: bytes
ETag: "1713"
CF-Cache-Status: HIT
CF-RAY: 60c70817a7b26a5d
alt-svc: h2=":443"; ma=86400
root@kali:~#
```

Figure 8: Baseline HTTPS

5.2 ICMP Block Rule

Rule **LAB2 BLOCK ICMP TO 1.1.1.1**: Action Block, Interface LAN, Direction In, Protocol ICMP, Source LAN network, Destination 1.1.1.1, Log enabled.

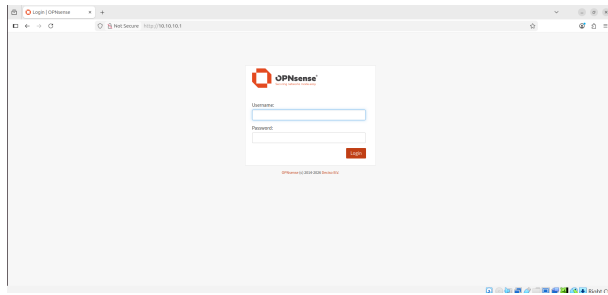


Figure 9: OPNsense login

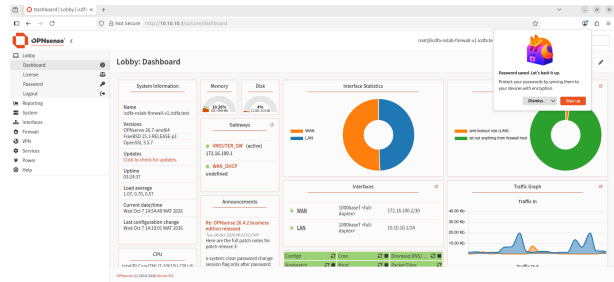


Figure 10: OPNsense dashboard

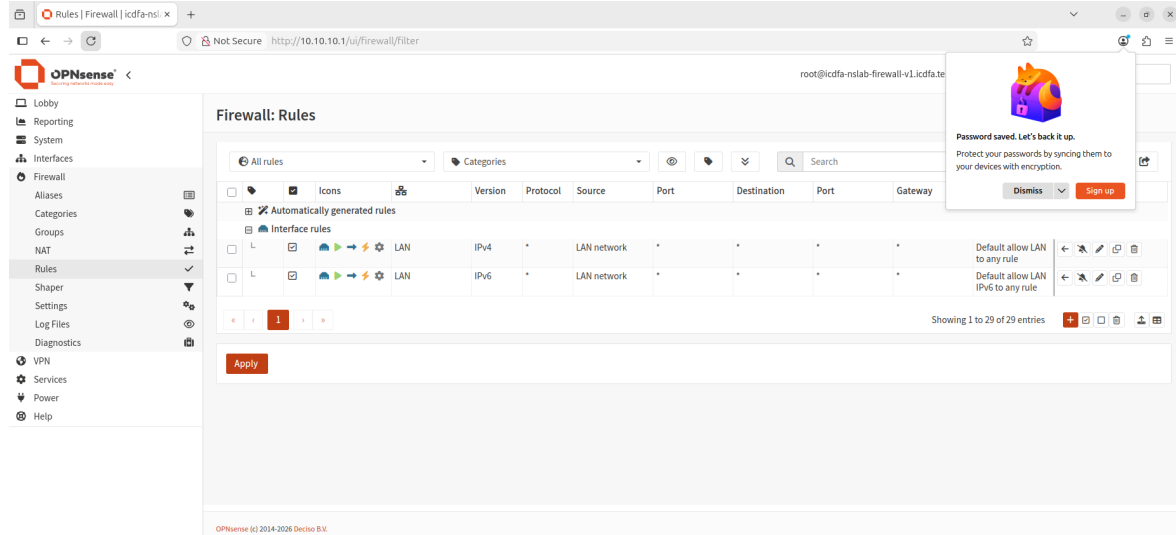


Figure 11: Default LAN firewall rules

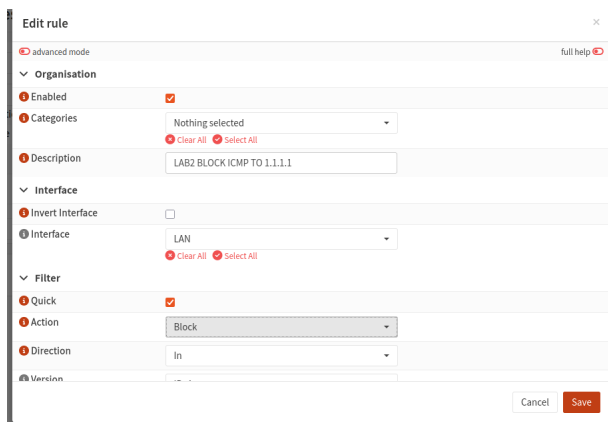


Figure 12: ICMP rule: organisation

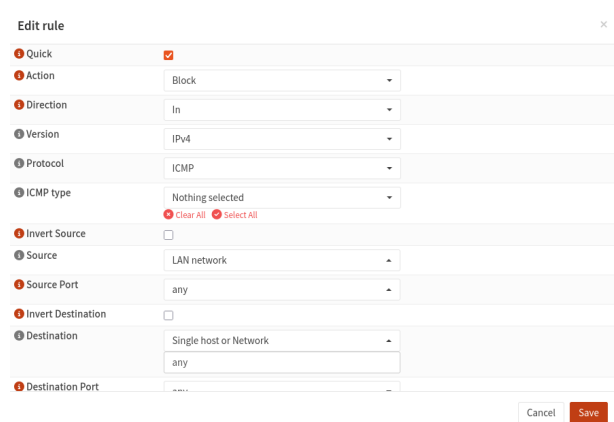


Figure 13: ICMP rule: filter

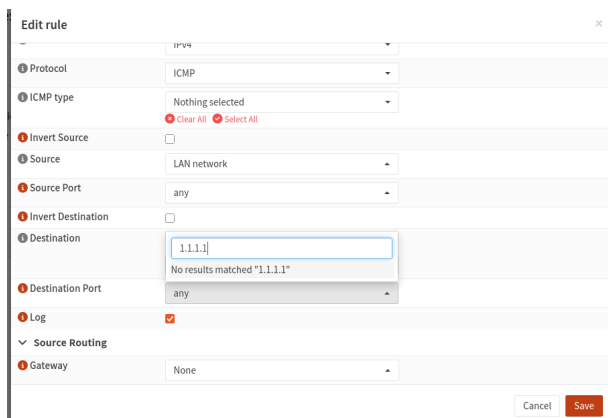


Figure 14: Destination lookup

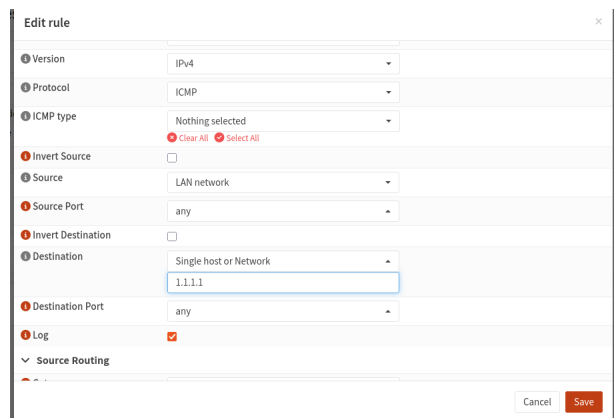


Figure 15: Destination set to 1.1.1.1

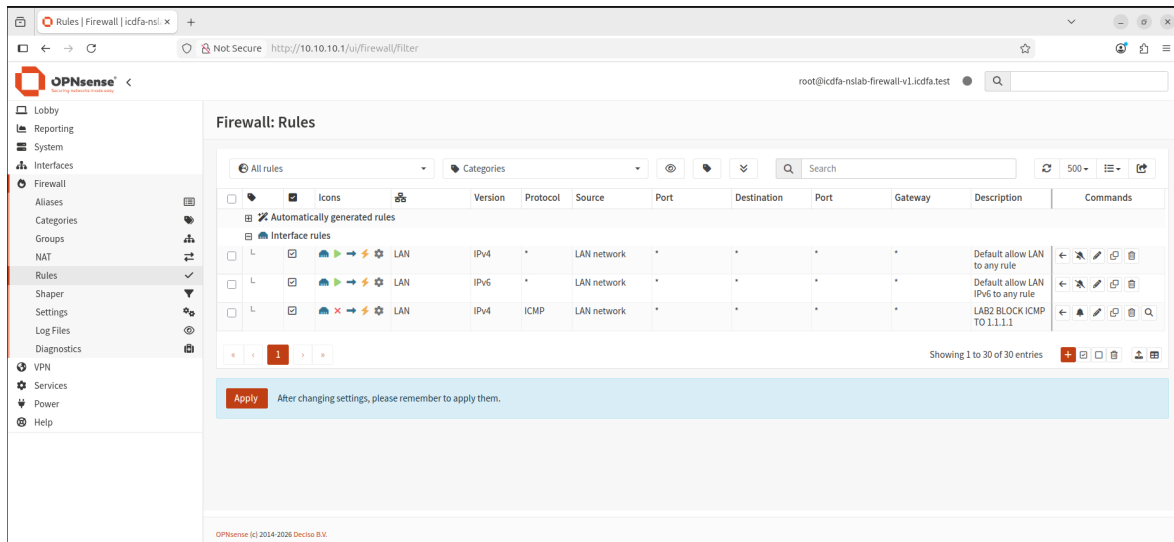


Figure 16: Rule list with changes pending apply

Ping tests while the rule was being configured (before the change was applied) still succeeded:

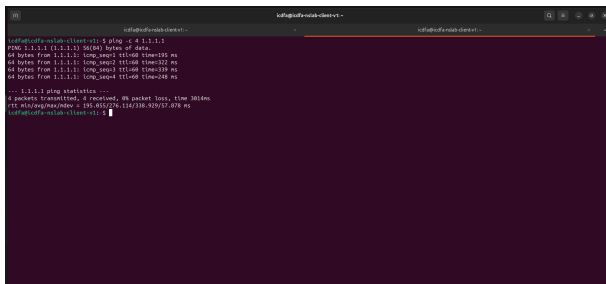


Figure 17: Ping test 1

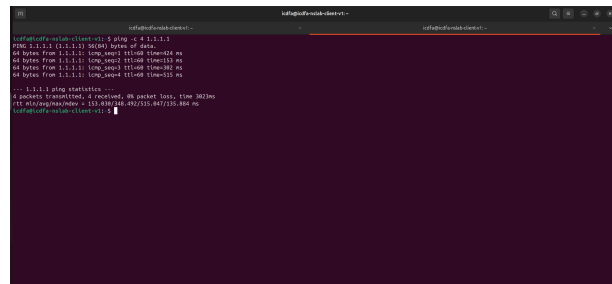


Figure 18: Ping test 2

After the change was applied:

`ping -c 4 1.1.1.1`

4 packets transmitted, 0 received, +4 errors, 100% packet loss

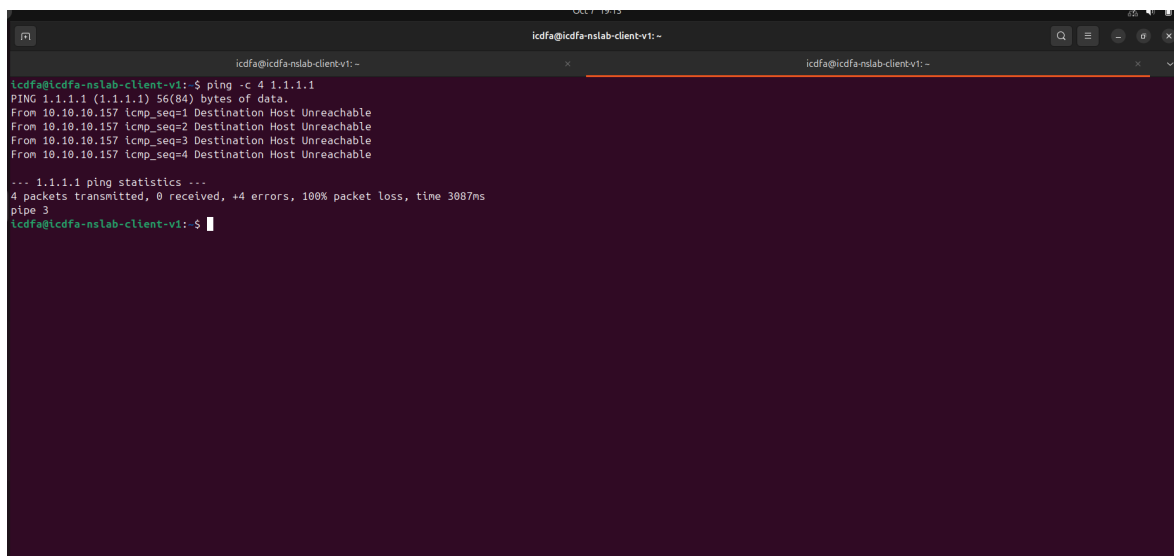


Figure 19: ICMP blocked after rule applied

Status: PASS - ICMP to 1.1.1.1 was blocked.

5.3 DNS and HTTPS after the ICMP Rule

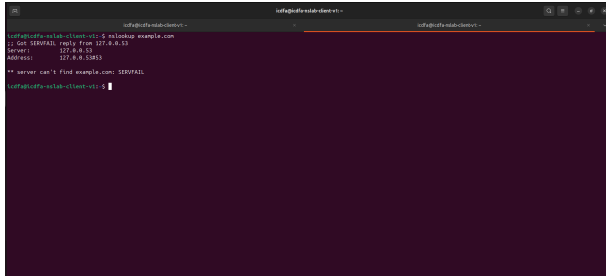


Figure 20: DNS: SERVFAIL observed (see Section 7)

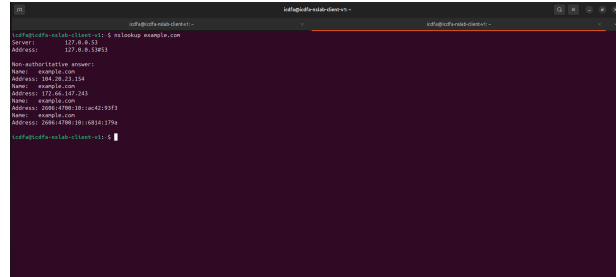


Figure 21: DNS: resolved

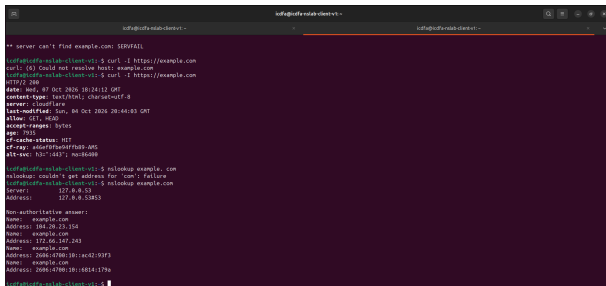


Figure 22: DNS/HTTPS recheck

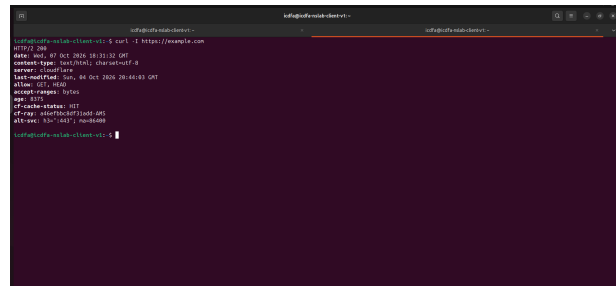


Figure 23: HTTPS working (HTTP/2 200)

DNS resolved and HTTPS returned HTTP/2 200, showing the ICMP rule did not affect other protocols.

Status: PASS

5.4 HTTP Block Rule

Rule **LAB2 BLOCK OUTBOUND HTTP**: Action Block, Interface LAN, Protocol TCP, Source LAN network, Destination any, Destination port HTTP (80).

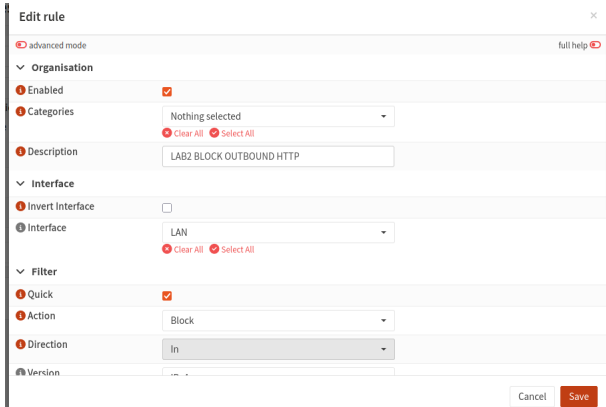


Figure 24: HTTP rule: organisation

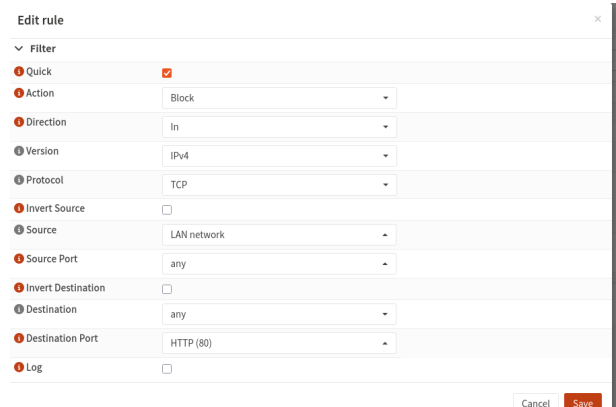


Figure 25: HTTP rule: filter (port 80)

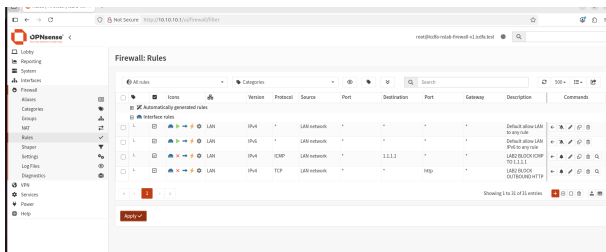


Figure 26: Rules list

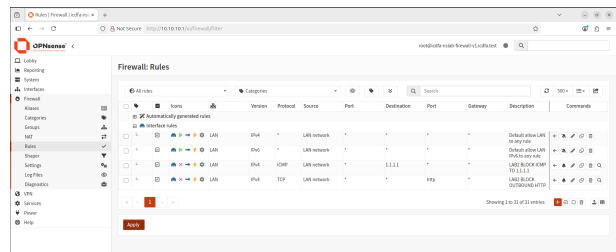


Figure 27: Rules list with both rules

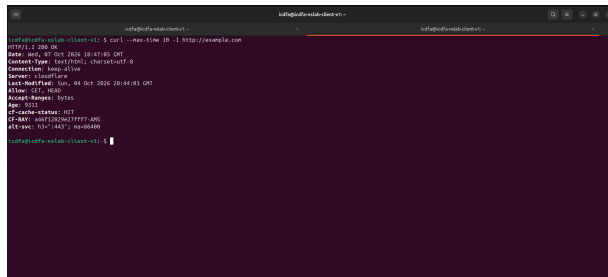


Figure 28: HTTP test

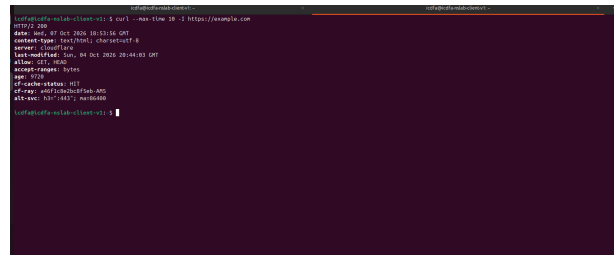


Figure 29: HTTPS test

HTTPS (TCP/443) continued to work. See Section 7 regarding the HTTP result.

5.5 Firewall Log Analysis

Reviewed via Firewall > Log Files > Live View.

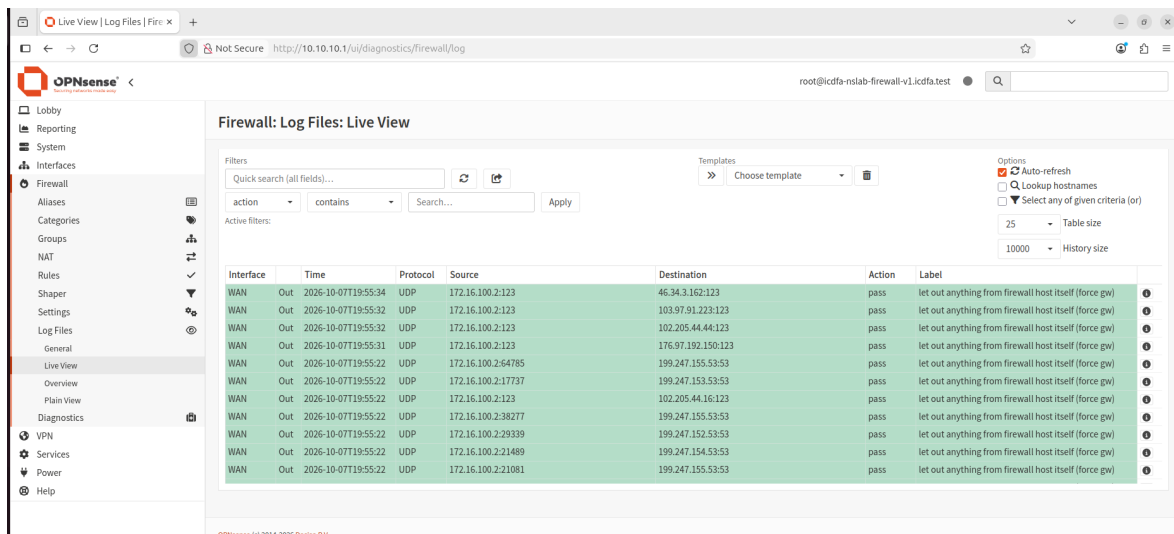


Figure 30: Firewall Live View

5.6 Restoration

Both lab rules were disabled (greyed out in the rule list) and the changes applied.

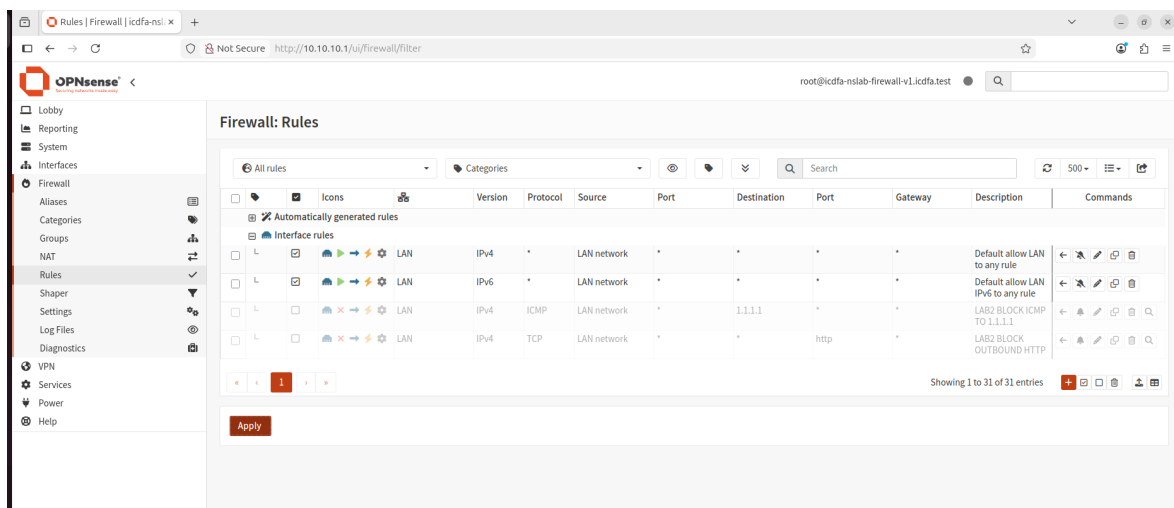


Figure 31: Rules disabled

6. Analysis Questions

1. Why should blocked traffic be logged?

Logs prove a rule was triggered and support troubleshooting, monitoring of policy effectiveness, detection of unwanted traffic, and incident investigation.

2. Why did DNS and HTTPS continue to work after the ICMP rule?

The rule matched only ICMP to 1.1.1.1. DNS (port 53) and HTTPS (TCP 443) use different protocols and ports, so they did not match.

3. Why can HTTPS be allowed while HTTP is blocked?

HTTP (TCP 80) is unencrypted; HTTPS (TCP 443) is encrypted. Rules are port-specific, so one can be blocked and the other allowed, encouraging secure communication.

4. What evidence proves the firewall processed the traffic?

Entries in Firewall > Log Files > Live View and the changed client results (ICMP loss after the rule was applied) show the firewall evaluated and acted on traffic.

5. Why restore connectivity at the end?

To return the lab to a known operational state so later labs are not affected by leftover restrictions.

7. Observations and Limitations

Recorded for accuracy, based on the captured evidence:

- **ICMP timing:** Pings succeeded until the rule was saved with the correct destination and changes were applied; blocking was confirmed afterwards.
- **Block response:** The client reported "Destination Host Unreachable" with 100% loss.
- **Transient DNS error:** One SERVFAIL was captured on the local resolver (127.0.0.53) before resolution succeeded on retry. An ICMP-only rule cannot block DNS, so this is most likely resolver or upstream timing rather than a firewall effect.
- **HTTP test:** The captured curl to `http://example.com` returned HTTP/1.1 200 OK, so the HTTP block was **not demonstrated** in the saved evidence. A likely cause is that the rule was saved but not yet applied. The HTTP rule also had Log unchecked. Re-test after Apply changes with logging enabled.
- **Live View:** The captured view shows allowed (pass) entries only; no blocked-traffic entries are visible. Enable logging on block rules and filter by action = block.
- **Restoration:** Evidence shows both rules disabled; a post-restoration ping screenshot was not captured.

Suggested re-test

```
ping -c 4 1.1.1.1           # expect 100% loss
curl --max-time 10 -I http://example.com # expect timeout
curl -I https://example.com  # expect HTTP/2 200
```

8. Conclusion

Firewall policies were implemented in OPNsense and validated from a Linux client. The ICMP rule blocked traffic to 1.1.1.1 while DNS and HTTPS stayed functional, demonstrating protocol-specific enforcement. The HTTP/80 rule was created and HTTPS remained available, and the rules were disabled to restore the lab. The exercise reinforced the need to apply changes, enable logging on rules of interest, and verify each policy with before/after tests.

Authorisation and Ethical Use

This lab was completed as part of the ICDFA Cybersecurity and Digital Forensics Programme. All testing was performed only against authorised systems in an isolated training environment.